# Exp07 — Where Does the Uop Go Between Decode and Retirement?

A resource-by-resource investigation of a Golden-Cove-style out-of-order backend

Deetya Ashish Mehta (Roll Number: CS23I1032)

October 2026

## At a glance

I built a cycle-accurate model of the stretch of the pipeline after decode and used it as an instrument: every time rename/allocate refused to take a uop, the model recorded *which* structure was full, *why* it was full, and *which older event* kept it full. The table summarises what the machine actually did; the rest of the report shows the evidence.

| Experiment | First thing to run out | Seen when | Root cause traced by the model |
|---|---|---|---|
| Long-latency load at ROB head | ROB (24/24) | cycle 5 | `LOAD` I0 completes at 22, so nothing behind it can retire |
| Narrow retirement (2 vs 4) | ROB | cycle 5 | retire rate below allocate rate, even with no slow instruction |
| ROB 48 / 96, PRF 32 | **PRF** (32/32) | cycle 6 | nothing is reclaimable while the load blocks retire |
| PRF 48 / 64, ROB 24 | still ROB | cycle 5 | extra registers sit idle |
| 10 loads in a row | load queue (6/6) | cycle 2, ROB 6/24 | LQ entries live until retirement |
| Branches waiting on a load | checkpoints (2/2) | cycle 2, ROB 7/24 | branches cannot resolve |
| **My own: stores queued behind a load** | **store queue (4/4)** | cycle 3, ROB 13/24 | stores *finished* at cycle 3 but cannot retire |

## 1. The model

**Machine.** Decode 6, rename/allocate 6, retire 4, ROB 24, 32 physical registers (PRF), IQ 12, LQ 6, SQ 4, 2 branch checkpoints. Latencies: ADD 1, MUL 4, LOAD 20 (STORE/BRANCH 1).

**What each cycle does, in order:** allocate, complete (and resolve branches), retire/reclaim, issue, decode. Since allocate goes first, anything freed in cycle $t$ helps allocation from cycle $t+1$. Each instruction carries timestamps for decode, allocate, issue, complete, retire/squash and reclaim, so "finished executing", "left the ROB" and "register returned" can be compared directly.

**Who needs what at allocation:** ALU/MUL: ROB+PRF+IQ. LOAD: +LQ. STORE: ROB+IQ+SQ (no register). BRANCH: ROB+IQ+checkpoint. Allocation is strictly in order, so one blocked uop blocks everything younger.

**When each resource is given back:** ROB, LQ, SQ at retirement; IQ at issue; a checkpoint when its branch resolves; a physical register when the *next writer of the same architectural register retires* (or at once when the writer is squashed).

**Stall diagnosis.** For every stalled cycle the model lists all missing resources, the primary one (order ROB, PRF, IQ, LQ, SQ, CKPT), the reason it is unavailable, and a root event found by following the oldest holder back through its unfinished producers (`results/stall_chains.csv`).

**Choices the brief left open** (fixed once, not tuned): a 12-entry decode queue between decode and rename; issue width 6; initial architectural values come from an always-ready ARF (a register costs a physical register only after it is written in flight); a predicted-taken branch ends its decode group; decode restarts the cycle after a recovery; no memory-dependence modelling.

## 2. Rename and allocation

*Workload:* the assignment's stream extended to 12 uops (`inv1_rename`), plus the same stream x5.

R1's journey through the machine:

| renamed by | physical reg | consumers (issue cycle) | returned to free list |
|---|---|---|---|
| I0 | P0 | I2 (3) | cycle 8, when **I3** retires |
| I3 | P3 | I4 (3) | cycle 10, when I7 retires |
| I7 | P7 | I8 (9), I9 (13) | cycle 17, when I11 retires |
| I11 | P11 | – | live architectural value |

- A destination gets a new register *in the cycle it is allocated* (I0-I5 take P0-P5 together at cycle 1).
- Which register does a reader see? Whatever the RAT held at *its* rename: I2 reads P0, but I4, renamed in the same cycle as I3, reads P3. One name, two registers, one cycle.
- P0's last reader issued at cycle 3 and its value was done by 7, but the register is returned at 8: reclamation is tied to the overwriter's retirement because until then a flush could still make the old value current again.
- Pressure: five architectural registers written 60 times peaked at **29 physical registers** in use (min. 3 free). The first stall in that run was the **IQ** (12/12 at cycle 3, ROB only 16/24), because dependent chains fill it, not the register file.

## 3. Different uops, different bottlenecks

| workload | stall at | blocked uop | ROB use | exhausted | duration |
|---|---|---|---|---|---|
| 10 independent LOADs | cycle 2 | I6 | 6/24 | LQ 6/6 | 22 cycles |
| 8 STOREs | cycle 1 | I4 | 4/24 | SQ 4/4 (PRF: 32 free) | 4 cycles |
| LOAD + 20 dependents | cycle 3 | I13 | 13/24 | IQ 12/12 | 20 cycles |

Free ROB space is not acceptance capacity: the uop must be *simultaneously* provided with every resource its class needs. Stores here never touched the PRF (32/32 free) and still stalled at ROB 4/24. The IQ case shows a third flavour: not a lifetime issue but a *dependence* issue, where twelve consumers wait for one load and occupy the scheduler without being able to leave.

## 4. Long-latency load at the head of the ROB

`LOAD R1` followed by 120 independent ADD/MUL uops (`inv3_longlat`).

```
cycle  1-4 : 6 uops allocated per cycle, ROB 6 -> 24;  I0 LOAD issues at 2 (due 22)
cycle  5   : ROB 24/24  -> FIRST allocation stall   (PRF 8 free, IQ 0/12, LQ 1/6)
cycle  6   : decode queue 12/12 -> FRONTEND stall
cycle  <=21: 23 younger uops finished; 0 retired; 6 issue slots idle every cycle
cycle  22  : load completes      cycle 23: retirement restarts (4/cycle)
cycle  49  : last stalled cycle  (alloc throttled to the retire rate for 27 cycles)
```

Causal chain: **frontend stall (6) <- decode queue full <- ROB full at I24 (5) <- 23 completed uops cannot retire <- oldest instruction I0 LOAD not complete until 22.**

The three events pulled apart on one instruction: I1 `ADD R2` issues at 2, completes at 3, retires at 23, and the *previous* R2 register P1 is returned only at 24 when its overwriter I7 retires. That is 20 cycles from "result ready" to "ROB slot free", and a stall that outlives the load itself: 18 of its 45 stalled cycles are the load, 27 are retirement bandwidth.

## 5. Retirement as a bottleneck

| run | cycles | stalled cycles | frontend-stalled | max done-not-retired | idle issue slots/6 while stalled |
|---|---|---|---|---|---|
| load workload, retire 4 | 57 | 45 | 41 | 24 | 3.9 |
| load workload, retire 2 | 84 | 67 | 60 | 24 | 4.5 |
| no-load workload, retire 4 | 41 | 29 | 25 | 19 | 2.7 |
| no-load workload, retire 2 | 66 | 49 | 42 | 20 | 4.0 |

Halving the retire width costs +47% and +61% time. The no-load rows are the telling ones: with nothing slow in the machine, 6-wide allocation into 4-wide (or 2-wide) retirement still fills the ROB at cycle 5 and stalls the frontend. Execution is idle because the units only see uops that already own a ROB slot; the machine is not short of execution, it is short of the ability to *hand resources back*.

## 6. Scaling the structures

Prediction made by counting before running: while the load blocks retirement nothing is returned, so the in-flight limit is min(ROB, PRF).

| ROB | PRF | cycles | first limit | stall cycles ROB / PRF |
|---|---|---|---|---|
| 24 | 32 | 57 | ROB (cycle 5) | 45 / 0 |
| 48 | 32 | 55 | **PRF (cycle 6, ROB 32/48)** | 0 / 43 |
| 96 | 32 | 55 | **PRF (cycle 6, ROB 32/96)** | 0 / 43 |
| 24 | 48 | 57 | ROB | 45 / 0 |
| 24 | 64 | 57 | ROB | 45 / 0 |
| 48 | 48 | 54 | ROB (cycle 9), PRF close behind | 15 / 19 |
| 96 | 96 | 54 | ROB (cycle 17) | 7 / 7 |

Enlarging the ROB did not relieve pressure, it relocated it: at ROB 48 the 32 registers all belong to in-flight versions (25 of them superseded and un-reclaimable). Enlarging only the PRF changed nothing because the ROB was the tighter structure. The best result (54 cycles) is within a cycle of load latency + 121/4 retire cycles: once ROB and PRF are generous the limit is **retire width**, a structure never varied in this sweep.

## 7. Physical-register lifetime

| case | stream | P(old R1) read by | returned | lifetime |
|---|---|---|---|---|
| A | `ADD R1; ADD R1; ADD R6,R1,..` | nobody | cycle 4 | 3 cycles |

| case | stream | P(old R1) read by | returned | lifetime |
|---|---|---|---|---|
| B | `ADD R1; LOAD R6,[R1]; ADD R1; ADD R8,R1,..` | the 20-cycle load | cycle 24 | **23 cycles** |

In B the overwriter I2 finishes at cycle 3 but cannot retire until the older load has, so the old version is pinned for 23 cycles. Case C (`LOAD R9` then 40 x `ADD R1`, ROB 64) writes *one* architectural register and still empties all 32 physical registers (stall from cycle 6). The number of live speculative *versions*, and how long retirement leaves them alive, sets the register need, not the architectural register count.

## 8. Misprediction recovery

Branch predicted Taken, actually Not-Taken. The listed instructions I3-I6 are the fall-through (correct) path; the Taken path W0-W4 (`ADD R7`, `MUL R9`, `LOAD R12`, `STORE [C]`, `ADD R13`) is the wrong one.

| | just before (cycle 3) | just after |
|---|---|---|
| rename table | R1->P0, R4->P1, R7->P2, R9->P3, R12->P4, R13->P5 | R1->P0, R4->P1 |
| ROB | 8/24 | 3/24 |
| free physical regs | 26 | 30 |
| IQ / LQ / SQ / CKPT | 5 / 1 / 1 / 1 | 0 / 0 / 0 / 0 |

Recovery restores the RAT from the branch's checkpoint, frees P2-P5 immediately, empties wrong-path entries from all four queues, flushes the decode queue and re-steers the frontend; correct-path decode is at cycle 4, so the stream takes 13 cycles against 10 with a correct prediction. The old mappings displaced by wrong-path writes are untouched, so no un-freeing is needed. When the branch instead waited on a load (`inv7b`), the wrong path ran 20 cycles: six wrong-path uops completed, a wrong-path load was still in flight, and LQ/SQ held 2 and 1 entries at the squash.

Normal path: allocate, execute, complete, **retire, reclaim**. Wrong path: allocate, execute/wait, **mispredict, squash, reclaim**: resources return at the squash, never through retirement.

## 9. My own bottleneck: store queue behind a load

**Prediction (before running).** *Resource:* the **store queue (SQ = 4)**. *Why:* program is `LOAD R1` followed by a repeating group `STORE, ADD, ADD`. The stores do not depend on the load, so they will execute in a few cycles, but SQ entries are held until retirement and retirement is blocked by the load. The fifth store should find the SQ full while the ROB is only about half used and registers are plentiful. This differs from the plain 8-store test of Section 3 because here the SQ is full of stores that have *already finished*. *Would prove me wrong:* a first stall on anything but SQ; SQ full while its holders are still executing instead of done; or the stall ending before the load returns.

**Observed (`final_sq_behind_load`).**

| | |
|---|---|
| first stall | **cycle 3**, blocked uop I13 (the 5th STORE), resource **SQ 4/4** |
| other resources at that moment | ROB 13/24, PRF 23 free, IQ 7/12, LQ 1/6, CKPT 0/2 |
| holders | I1, I4, I7, I10 (stores) – all four **complete** at cycle 3-4 |
| duration | 22 stalled cycles (3-24), all attributed to SQ |
| release | I0 completes 22, retires 23 with I1; I13 allocates at 24 |

Chain, as the model prints it:

```
Immediate resource : SQ 4/4 (I13 blocked)
Why unavailable    : SQ entries are held until retirement; the four holders are DONE, not running
Earlier event      : retirement blocked by I0 LOAD, completes at cycle 22
```

Prediction confirmed on all three counts. The notable detail is the change of reason inside one stall: at cycle 3 the oldest store was still *executing*; from cycle 4 on it was *done and waiting to retire*. The SQ was never short of execution, only of retirement. Also, ALU uops behind the 5th store (I14 onward) were blocked though they need no SQ entry, and the ROB peaked at 13 of 24 for the stall's duration.

## 10. Short answers

**A.** A ROB slot only gives an uop a place in the order; it also needs a version of its result (PRF), a scheduler slot (IQ) and, by class, an LQ/SQ/checkpoint slot. Section 3: the 7th load had 18 free ROB slots and no LQ slot.

**B.** R1 -> P0, P3, P7, P11 (Section 2); each old register is reusable once the next R1 writer retires (cycles 8, 10, 17).

**C.** I1 `ADD R2` completed at 3 but sits in the ROB until 23; releasing it earlier would discard the only record needed to roll back if the older load faulted or the path were squashed.

**D.** Behind a slow oldest instruction, up to 23 finished uops accumulate; with retire width 2 the pile stays high, and during stalls 4 of 6 issue slots are idle.

**E.** `inv2a` cycle 2: ROB 6/24, PRF 26 free, but I6 blocked on **LQ 6/6**. Also `final_sq` cycle 3: ROB 13/24, **SQ 4/4**.

**F.** No: ROB 24 -> 48/96 moved the limit to the PRF (32 in-flight), ROB 48+PRF 48 was shared, and past that retire width set the 54-cycle floor.

**G.** I0 LOAD (due 22) -> retire blocked -> 23 finished uops stuck -> ROB 24/24 (cycle 5) -> decode queue 12/12 -> frontend stall (cycle 6).

**H.** Discard: younger ROB/IQ/LQ/SQ entries, in-flight wrong-path execution, their destination registers, younger checkpoints, decode queue. Restore: RAT from the checkpoint, fetch PC.

**I.** Each structure adds area, power and latency, and each is only useful while the structures around it keep up. Our ROB-scaling run found the PRF, then retire width, within three steps, and neither branch accuracy (`inv7b`: 20 cycles of discarded work) nor data dependences (a load plus its consumers filling the IQ at ROB 13/24) can be fixed by capacity.

## Appendix – Reproduce and limits

```
cd "Assignment 7/src" && python3 run_all.py
```

Outputs: `results/traces/*.txt` (cycle-by-cycle trace, causal chains, per-instruction and register-version tables), `summary.csv`, `per_cycle.csv`, `per_instruction.csv`, `stall_chains.csv`, `preg_lifetimes.csv`, `inv5_grid.csv`, plots in `results/plots/`, inputs in `inputs/`. 23 runs, all logged.

Limits: LQ/SQ entries are freed at retirement (real stores hold SQ entries until the cache write), no memory disambiguation or port-level model, checkpoints are a counter, decode-queue size and issue width are my assumptions (they change *when* the frontend stall shows, not which resource is exhausted first).

## Plots

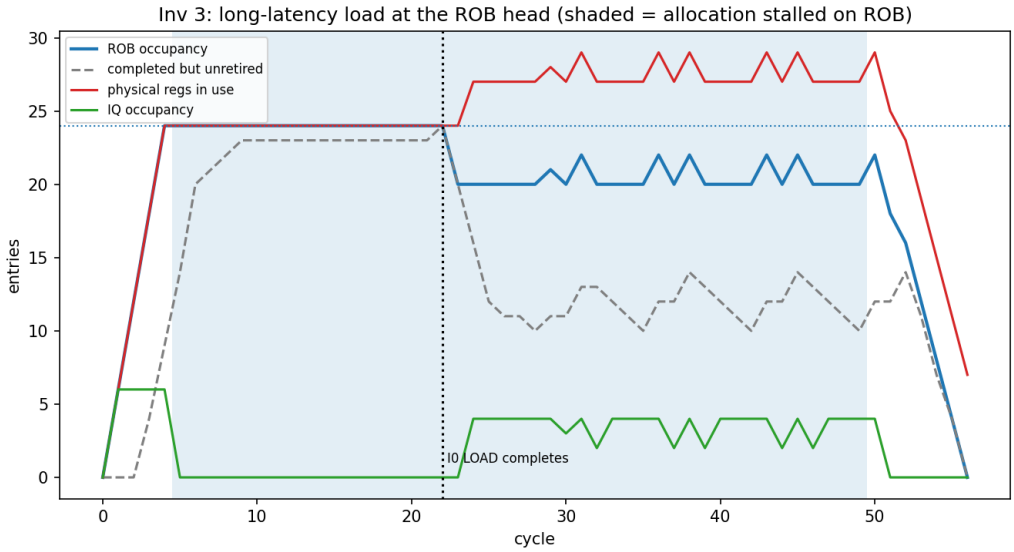



Figure 1: Long-latency timeline

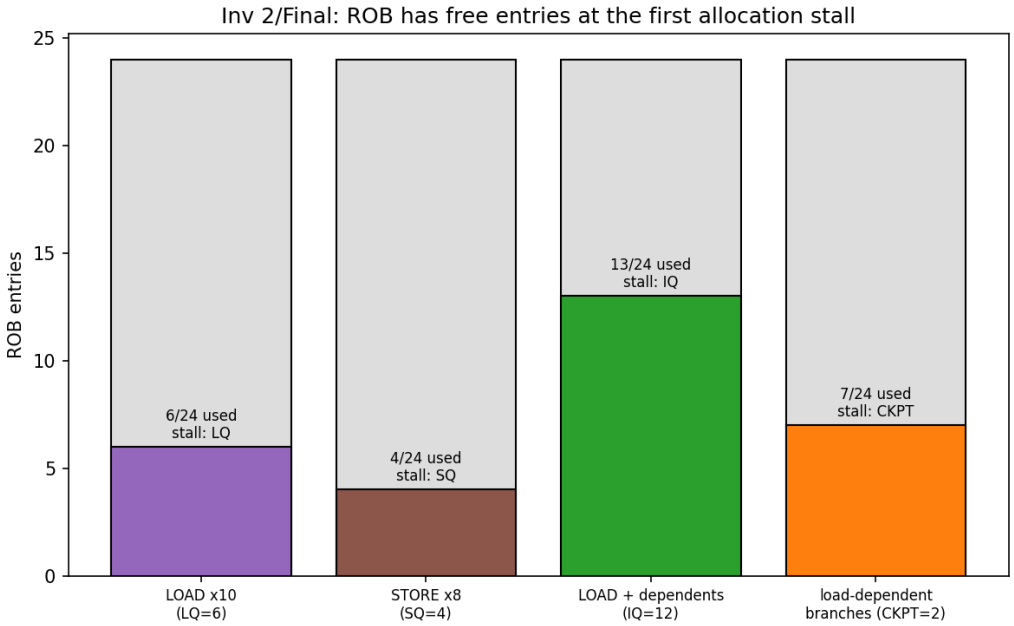



Figure 2: ROB headroom at first stall




Figure 3: Retirement width

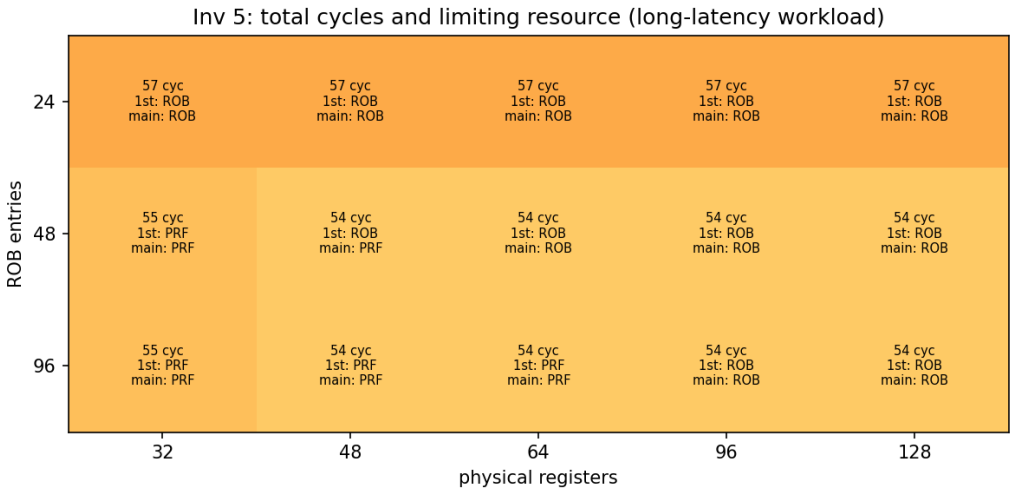



Figure 4: ROB x PRF grid

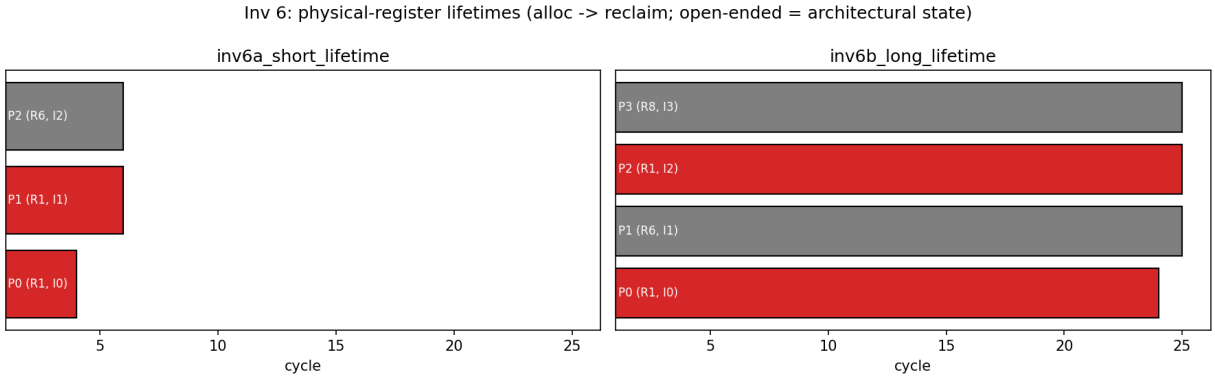



Figure 5: Register lifetimes

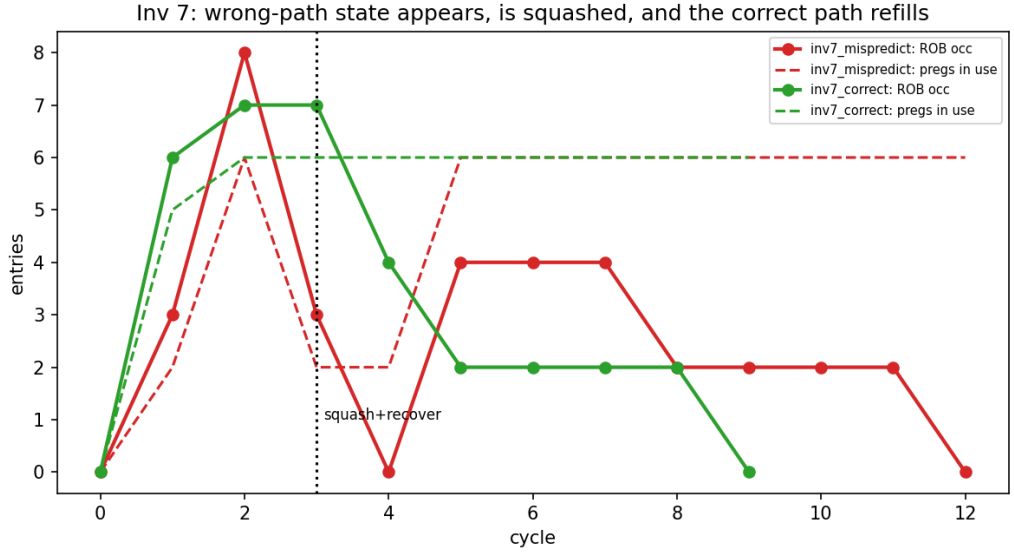



Figure 6: Recovery

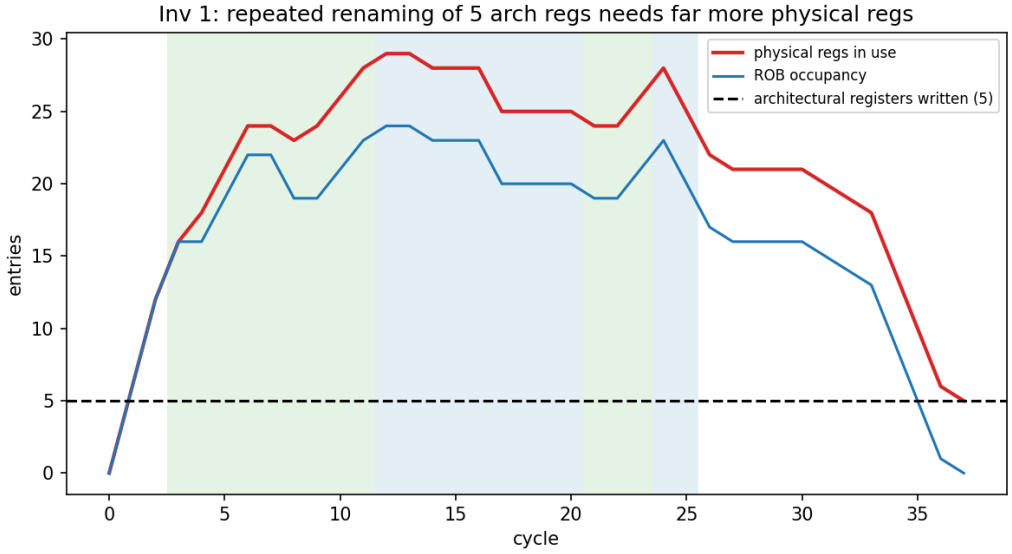



Figure 7: Rename pressure